Proc. of the International Conference on Electrical, Computer and Energy Technologies (ICECET 2022)
20-22 June 2022, Prague-Czech Republic

Sharing Fungible Assets Across Polkadot Paraverse

Dušan Morháč
Faculty of Informatics and Information Technologies
Slovak University of Technology in Bratislava
Bratislava, Slovakia
xmorhac@stuba.sk

Viktor Valaštín
Faculty of Informatics and Information Technologies
Slovak University of Technology in Bratislava
Bratislava, Slovakia
viktor.valastin@stuba.sk

Kristián Košťál
Faculty of Informatics and Information Technologies
Slovak University of Technology in Bratislava
Bratislava, Slovakia
kristian.kostal@stuba.sk

***Abstract*—Fungible asset sharing in cross-blockchain technology is becoming more popular and more developed. Blockchain interoperability at the beginning of blockchain technology was practically non-existent. The first blockchains did not have any communication possibilities with the other chains. Cross-blockchain asset sharing became possible with the recent rise in popularity of blockchain interoperability. Newly developed blockchains are slowly implementing cross-blockchain sharing capabilities as standard features. This interoperability allows for many new significant use cases, such as in the gig economy or decentralized finances. This paper proposes a cross-blockchain fungible asset sharing system that will be able to transfer assets across specific Polkadot paraverse ecosystem Parachains.**

***Index Terms*—Blockchain, Cross-blockchain, Asset sharing, Kusama, Polkadot, Fungible assets**

I. Introduction

Blockchain interoperability is becoming a standard for newly built blockchain networks these days. The first blockchains have been isolated, unable to communicate with each other. Further, they were competing for users or developers. Asset transfers have been possible only between users of the same blockchain network. This is slowly being solved, with blockchain interoperability increasing in popularity. Users can now share assets between compatible blockchains using specific protocols and bridges or share them by backing tokens from the original chain in a one-to-one relation with tokens from the destination chain [11].

There are many interoperability solutions. One of them is bridges. As the official Ethereum website [5] explains, bridges in a network work like bridges in the real world. They connect two networks, allowing for cross-blockchain transfers and communication. Some bridges are centralized, while others are decentralized. Centralized rely on trusted entities, while decentralized are constructed from smart contracts that handle events without any third-party involvement. Bridges come with advantages such as:

- Lower transaction fees
- Ability to interact with applications on different networks
- Possibility of owning native assets from other networks
- Exploration of blockchain ecosystems

As [21] mentioned, multiple blockchain interoperability solutions have different weaknesses or limitations. Sidechains (RSK [22]) - These were mainly implemented for cryptocurrency exchange and did not provide many other valuable functionalities. Routers (Anlink proposal [1]) - Most proposals were never implemented. In addition, the network and node architecture would have to be changed to act as a router. Smart contracts (Autobahn Network [2]) - This feature still needs further research for weaknesses Industry solutions (Polkadot [19], Cosmos [8]) - They rely on trusted entities for validation. If they are taken down, the network is vulnerable.

As [6] mentioned, we are currently capable of transferring tokens or messages across blockchain networks. However, they proposed two ideas for cross-blockchain data transfer.

- Transfering native data from the blockchain (storage) across different blockchains.
- Storage for blockchain data that is shared between blockchains.

There is also a proposal for Multi-blockchain Architecture [26] that allows for the interoperation of heterogeneous blockchains by connecting them through so-called coordination nodes. Individual blockchains trust these nodes. They are connected, forming a peer-to-peer network that holds information on the routes to each blockchain, the identity authentification of each blockchain, and other helpful data, essentially creating multiple blockchain systems. According to [25], during the cross-blockchain transaction, data are primarily stored in the cross-chain center. They explain that cross-chain transactions between two networks, without involving a cross-chain center, can increase the throughput of the transactions if the cross-chain center that networks rely on has surged. The proposed solution is called a Virtual chain. The virtual chain acts as a storage for heterogeneous cross-blockchain transactions. Many protocols define how cross-blockchain transfers should be executed. The paper examines the protocols proposed by the Polkadot network.

Using particular protocols like Horizontal Relay-routed Message Passing, [17] also known as HRMP, it is possible to define a format in which messages between different

978-1-6654-7087-2/22/$31.00 © 2022 IEEE

parachains that share the same Relay chain are transferred. However, this protocol is only an early stage of the XCMP protocol and is more resource-dependent because it stores all messages in Relay chain storage [17]. The cross-chain message-passing protocol, or, in short, XCMP, is said to deprecate the HRMP protocol once it is developed and will mainly differ in resource management. XCMP will only need to store a small amount of metadata on the Relay chain compared to HRMP, which has to keep all messages on the Relay chain. There are multiple proposals for cross-blockchain asset-sharing protocols. One of them is a proposal for the Burn-to-claim protocol [16], for example. This protocol works by launching a function that sends assets to a so-called "burn address," where they can never get recovered from, and letting the function generate proof of locking them. The burn address does not have a private key, so the assets are locked forever. The destination chain should have a function to verify burn-proof that re-creates burned assets. The cross-blockchain interoperability also depends on the network architecture. This paper examines Polkadots architecture, which is the foundation of the network solution.

Currently, the Polkadot ecosystem consists of a Relay chain, Parachains, and Parathreads. The Relay chain is the leading chain responsible for Parachain slot auctions, governance, staking, and security. Parachains, on the other hand, can have specific use cases, run particular apps, and can have specific fungible or non-fungible assets. As long as Parachain owns the slot won in a candle auction supported by crowd-loan, they are continuously connected to the Relay chain. Parathreads, similarly to Parachains, are used for specific use cases. However, they use the parachain slot in the "pay-as-you-go" style. They are only woken up when they need to fetch required information, execute functions or smart contracts, and transaction fungible or non-fungible assets.

Fungible assets or tokens can be divided into smaller particles and larger quantities. They do many things, mostly paying transaction fees, bidding, staking, voting, governance, or transactions. For example, the DOT token is a native fungible token in Polkadot. On the other hand, non-fungible assets are unique and cannot divide into smaller particles. They can represent the ownership of something, for example, in-game items.

Cross-chain asset sharing can have a ton of potential use cases other than transferring assets across different blockchains. It can be used for gig services, where freelancers provide a service for a fee. This transaction can be done with the smart contracts in use. Cross-chain asset sharing can also be used in ideas such as Vickrey's cross-blockchain auction proposal [12]. The idea is that users on blockchain A can create an auction for a specific asset, and users from blockchain B can bid on it. After the auction, the winning bidder from blockchain B pays the second largest bid in requested currency and receives the asset from the blockchain A auction starting user. Cross-blockchain asset sharing can also be used, for example, in the sharing economy.

The sharing economy, in general, is gaining popularity. Nevertheless, few people are used to sharing their assets for a profit with strangers. These sharing services can propose issues such as trust issues or issues with users intentionally triggering transaction refunds during service by the service provider without the provider knowing. On the other hand, it can also propose benefits. For example, decreasing pollution if it is a car-sharing service [24], more freedom for freelancers, or different meaning to the employee and employer relationship. Keeping benefits while solving the mentioned issues can be done using blockchain technology. Transaction data are permanently stored; if the transaction was processed, it could not be intentionally refunded by a user who ordered the service. The user can get his funds back only if the service provider decides to refund the payment due to some issue.

The use of blockchain technology in the sharing economy has also proved to be the solution to some of the security flaws centralized systems that provide sharing services have. Many centralized sharing services have to deal with possible attacks that can create overall problems with the stability of services or denial of service attacks that can collapse the entire service. Using blockchain technology, this is not possible because the data are distributed and processed across the network [14]. All the data put into this network are permanently there and cannot be changed. There is no risk of tampering with user data unless they want to change it. If they decide to change it, they will have to inform everyone on the network. Due to the append-only nature of blockchain, this increases network transparency and decreases the chance of user data change without the user knowing.

To address the cross-blockchain asset-sharing issue, the paper proposes a system that can transfer fungible tokens across specific parachains in the Polkadot paraverse ecosystem. The project has the following goals:

- Safety, not relying on any entity or any central organization
- User friendly, so that regular users can also use cross-blockchain asset transfers
- Open-source, so that community can contribute and everything is completely transparent
- Simple code, so that developers can easily understand code and continue developing it

II. Related Work

This section will cover similar projects currently proposing innovations in blockchain interoperability, allowing simple cross-blockchain asset sharing. The first project with a concept similar to octopus is the Octopus network [15]. It works on a very similar principle as an octopus, from which it also derived its name. Octopus's legs have multiple nerve systems connected to the brain; however, they can make decisions independently, essentially creating systems that work independently but are connected to one central controller - the octopus's brain. The project has a very similar approach. One main chain is also running along with a NEAR chain, which brings a marketplace where applications running on different chains, also called Appchains connected to this

network, can lend cost-effective security. This means that compatible Appchains can communicate with either the NEAR protocol, the IBC pallet if they are IBC compatible, or by Rainbow bridge if Appchains are on Ethereum. This is an excellent example of blockchain interoperability that allows simple cross-blockchain asset sharing. The ecosystem has its fungible token, called OCT. It is used for staking, voting, and governance. The mainnet is already running, and its interface can be seen in the Figures 1 and 2.

There are currently three Appchains called "myriad," "deep" and "debionetwork" and three bridges to them from the NEAR network running. Current Appchains statistics can be browsed in the Appchains overview. Auditing and registered Appchains that are incoming to the system can also be seen there. Cross-blockchain asset transfers from the NEAR network to running Appchains can be performed in the bridge overview section.

The second exciting project that allows for cross-blockchain asset sharing is XCLAIM and its Polkadot to Bitcoin bridge called Kintsugi [11]. It works by backing up Bitcoin's natural BTC token and minting a token called kBTC that has one-to-one value with BTC and can be used for various tasks, from which two are to Burn token for Kusama's KSM token or redeem the kBTC token back to BTC backed token. These operations are made with a set of smart contracts called:

- Issue, to wrap BTC token and mint kBTC
- Redeem, to redeem kBTC token back to BTC
- Burn, to burn kBTC token in exchange for KSM token

There are also other useful features that this project proposes, as can be seen in the Figures 3 and 4.

Kintsugi currently offers a testnet preview of features their project will offer. These are the options mentioned with backing up tokens using smart contracts. However, in the transfer section, they can do cross-blockchain transfer, where they currently offer transfer from KSM to Kintsugi assets and another way around. Their mainnet currently offers only regular transfers of kBTC, KSM, and Kint tokens with bridge features marked as coming soon.

The next project demonstrating blockchain interoperability and the ability to do cross-blockchain asset sharing is a multidirectional bridge from Moonbeam to Ethereum. It was developed by Chainsafe and is called Chainbridge [13]. The pros this bridging technology brings are: it can share assets that are both fungible and non-fungible; it is compatible with different chains that are Substrate-based or with Ethereum Classic and Ethereum; it is multidirectional, which means there is no need to construct two bridges to have communication flowing both ways; it is modular, and the security is constructed to prevent data fraud or fund manipulation by hackers.

As [13] mentioned, Chainsafe is a protocol that defines message passing at its core. There are three roles. Each role has a self-explanatory name and is performed by so-called relayers that need to be trusted: Listener's role is to construct the message extracted from the original chain. Routers role is routing message from origin to destination chain Writers role is to extract the message and submit it on the destination chain

Each side of the bridge has a set of smart contracts that serve different functions. There are the following:

- Bridge contracts take care of creating deposits, starting transactions, and are interacted with by users
- Handler contracts create a record from validated data that the user provides
- Target contracts are contracts that are interacted with on each end of the bridge. These can be imagined as structural poles that hold a bridge to the ground on both chains and receive or transmit critical data.

Chainbridge technology has some disadvantages. As mentioned before, Chainbridge is currently relying on the trust of the relayers. The system is set to protect users by making fraud and transaction manipulation not worth tampering with. It cannot guarantee this, however. The use of native asset-sharing protocols can solve this problem.

By comparing Chainbridge technology with Kintsugi, they solve the same problem with a different approach. While Chainbridge relies on relayers' trust and is a working solution, Kintsugi is still only available in the test net and depends on a safe mechanism for backing tokens so that they cannot get stolen by hackers. Chainbridge can also send fungible and non-fungible token types compared to Kintsugi, which can only transfer fungible tokens now.

III. Design

Using XCMP native asset sharing support can solve problems with trust. As the XCMP protocol is currently under construction and is not yet released, this proposal will use the already mentioned very similar protocol called HRMP. This protocol proved to be a viable and working native asset sharing replacement until XCMP is released. To begin the development of the solution, it is necessary to understand the logic of a few segments of the Polkadot ecosystem. These are Relay chains, Parachains, Validators, and Collators. These segments will be mentioned throughout this section. The first step is installing the Rust language [23], Polkadot is mainly programmed in. Rust is a typescript language in which many new blockchains are programmed, and the system must be installed to compile Polkadots code. The version used is nightly because the code uses the latest features.

The next step is to have the Relay chain running. This node is called the Rococo local test net node. It is already programmed in Polkadot's official Github repository [20]. After successfully installing the Polkadots Relay node, the next step is to generate raw chain spec for the Rococo local test net Relay chain. After generating chain spec, it is time to connect validators to Rococo local test net Relay chain and launch it. Example validators are also programmed on Polkadots Github [20]. Hence to demonstrate cross-blockchain communication, it is necessary to have two Parachains running. They should have at least three validators in the Rococo local test net Relay chain for less bottleneck. In a real network, the proposed number of validators to Parachains is 5:1 [18].

Once the Rococo local test net Relay chain is created and the validators are attached, it is time to have the Parachains

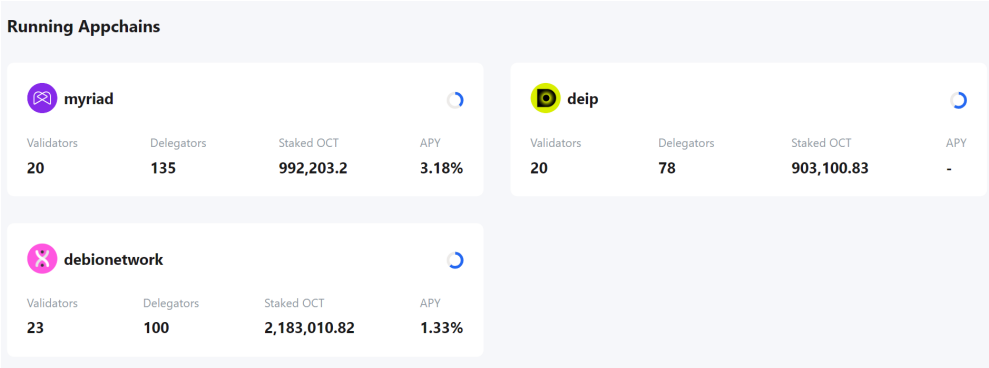

Fig. 1. Octopus network Appchains overview

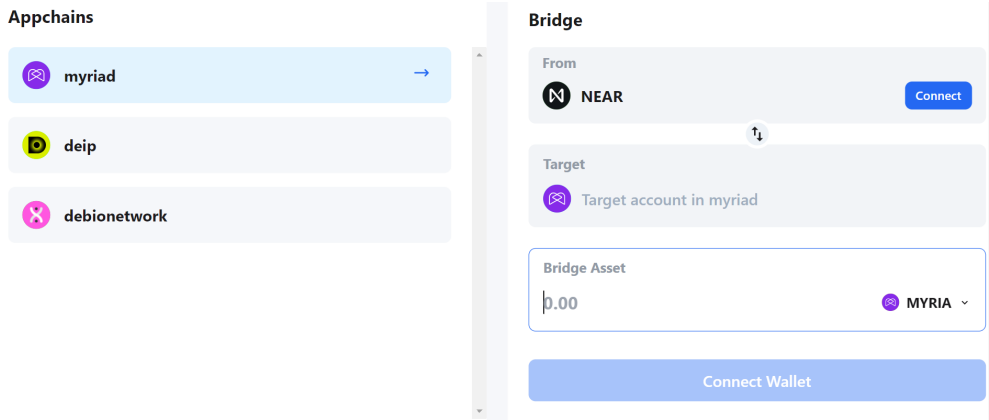

Fig. 2. Octopus network Bridges overview

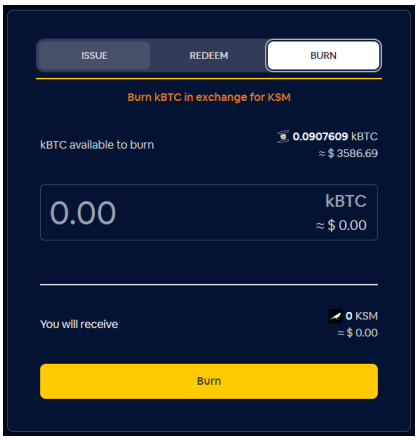

Fig. 3. Kintsugi Bridge section overview

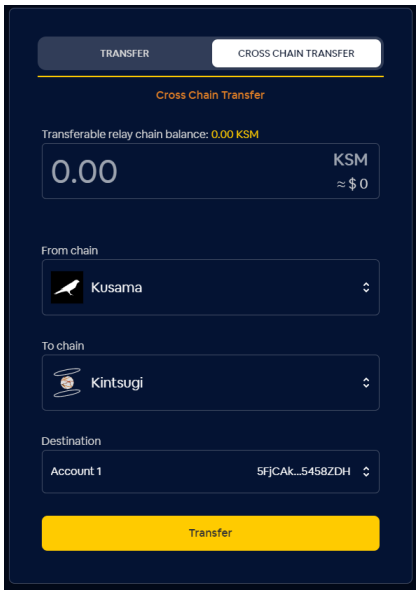

Fig. 4. Kintsugi Transfer section overview

running. The minimum amount of Parachains needed to run for cross-blockchain asset transfer is two. The Basilisk node from the Galactic council is the first Parachain suitable for this task [4]. This node contains an essential cross-blockchain sharing capable network startup configuration that can be used to launch the network later on. The second parachain is the Karura node from the Acala network [10]. This is due to the already mentioned network configuration file from Basilisk, which requires the Karura node from the Acala network to be compiled. Both Parachain nodes are also reasonably documented and frequently updated. After installing the Parachain

nodes, it is necessary to reserve paraIDs for them on the relay chain. This is done by winning the auction for the Parachain slot in a real network.

After successfully reserving slots for Parachains, it is time to generate their chain specs, in which they also have their Parachain ID defined. The process is similar to Relay chain spec generation. Next, it is necessary to generate a Web assembly file called Wasm. This file specifies the Parachain validation logic to the Relay chain. Once Wasm files for both Parachains were generated, the genesis state file was the last type of file that needed to be generated. Genesis file specifies Relay chain genesis state Parachain. Each Parachain trying to connect to the Relay chain needs to start with the genesis block. The genesis block is the first block on the blockchain. As [14] explained, the genesis block is block number 0. This means there cannot be any blocks before it, and each block can be traced through the previous block history since each block is connected to the preceding one. This means that Parachains attached to the Relay chain cannot have any prior history before initialization by a specific function. As [9] explained, this feature is expected to be developed eventually.

Parachains cannot operate without collators. They need to be connected to each Parachain before Parachains are assigned to the Relay chain. Polkadot explains [7] that Collators:

- Collect transactions from Parachain users and propose new block candidates
- Create state transition proofs from block candidates for Relay chain validators
- Are not required to secure Parachains since invalid blocks will get rejected by validators
- Are essential for cross-blockchain message passing (they store full node of Relay chain, so they are aware of other collators)

Collator examples are already programmed in both of the Parachain nodes. After launching and connecting them to their Parachains, Parachains need to be initialized on the Relay chain to start producing blocks. Parachain is initialized from the Relay chain using a special sudo function. This function is called ”parasSudoWrapper.sudoScheduleParaInitialize(id, genesis)”. The function requires the parachain ID we reserved, the genesis file and the wasm file. Providing the required information results in a new Parachain appearing in the Parachains section after a while.

This is all that must be set before the solution can run locally. It may be inconvenient, but later versions can bring real network deployment and much more convenient usability. Currently, there is a configuration startup file [3] that will create a network for us with the following parameters:

- Rococo Relay chain with eight validators
- Basilisk node with two collators
- Karura node with one collator

This configuration file will also regenerate required chain specs, genesis files, and wasm files on each network startup. After the local network is launched, the examination and launching of the solution can be done. The solution provides the user with a few different use cases:

- User can create a channel from chosen Parachain A to B and vice versa.
- User can create an XCM transaction from Relay chain to Parachain
- User can create an XCM transaction from Parachain to Relay chain
- User can create an XCM transaction from Parachain to Parachain
- User can remove channels they created

All this while keeping the user interface as simple as possible for quick operations. The HRMP channel is created by a specialized function called ”parasSudoWrapper.sudoEstamblishHrmpChannel(sender,recipient, max_capacity,max_message_size)”. The call requires specifying sender and recipient paraIDs. This call needs to be called twice with switched parameters to open multidirectional communication between Parachains. Currently, these channels can only be opened by wrapping the mentioned function with the sudo command and from the Relay chain. In a real network, the channels are opened by the council.

The cross-consensus message format [17], also called XCM, will be used to transfer assets through these channels. Each node has different functions or parameters to perform XCM transfers:

- Relay chain node can transfer XCM transactions through function ”xcmPallet.reserveTransferAssets”
- Basilisk and Karura node can transfer XCM transactions through function ”xTokens.transfer”, each node has different required parameters.

Closing Parachain channels is also done similarly to creating channels by calling a special function ”hrmp.forceCleanHrmp”. The figurerefdiag5 explains how the solution currently works.

Polkadot network’s basic concept contains a Relay chain, Validators, Parachains, and Collators. The added functionality of the solution is a simple user interface for opening HRMP channels across Parachains that are currently running. Another benefit is the simple transfer of assets or messages through these channels created by the user. Users can also choose to remove channels they have created. These channels are defined and created by the HRMP protocol and user input details. HRMP protocol solutions work by creating a one-way channel from each origin side to the destination Parachain. These channels will allow for cross-blockchain communication between the two Parachains. It will be possible to transfer tokens or messages in a multidirectional way. HRMP protocol is not very sophisticated in terms of storage. It will be deprecated in favor of XCMP in the future. XCMP will only need to store metadata on the Relay chain while HRMP has to keep all messages on the Relay chain.

IV. Test scenarios

Two tests were designed to analyze the time required for the task to be completed. The first test analyzed the time required

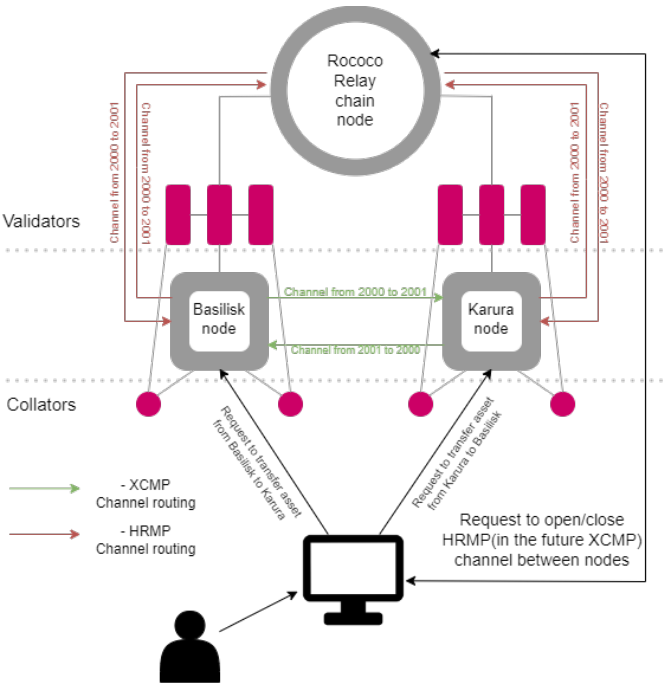

Fig. 5. Solution graphic representation and overview

to complete the XCM transaction in all three scenarios the application offers. The second test was to determine how intuitive the application is by giving ten users the task to send a transaction of their choice. The test was conducted with people of different age categories and IT skills. As explained in the introduction, the goals were safety, a user-friendly interface, and open-source, straightforward code.

The solution was developed and tested on a Huawei Matebook D14 running Ubuntu 21.10, with eight gigabytes of DDR4 random access memory and 256 gigabytes of a solid-state drive. The version of the Polkadot repository solution created is ”v0.9.19”.

V. Results

The design picked had the optimal results regarding the time required to complete the task. All tested designs showed that anyone can complete a task successfully and that a simple user interface can make cross-blockchain transfers more accessible to people. The first results of the test are shown in Figure 6. The graph explains that transfer times were mostly consistent, but there were some spikes. These spikes were mostly due to hosting networks locally, and they should not appear in a real network scenario. The fastest transfer is in scenario Relay chain to Parachain transfer. The widest curve in a vertical way is in the Parachain to Relay chain scenario. These transfers took approximately anywhere between ten and forty seconds. Compared to non-native protocols, native protocols are very reliable in spikes, time to complete, and completion.

The second test showed positive results regarding time gaps between different IT skills of users. The results can be seen in TableI. As the Table explains, the best time to send a

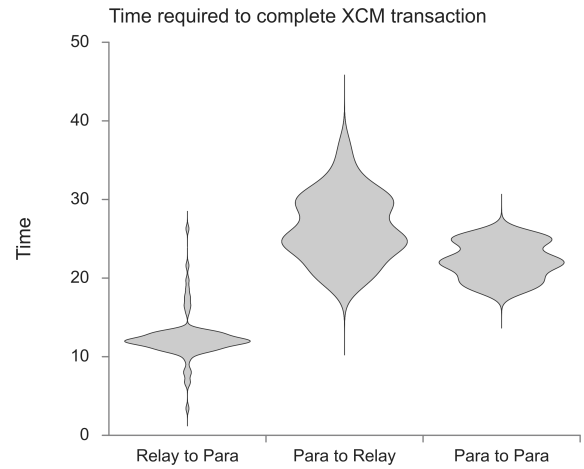

Fig. 6. Resulting in times to complete a transaction in three native XCM scenarios

transaction was 22.28 seconds. This time was achieved by the user with the most IT skill. This means that experienced users can adapt to new applications relatively quickly. The worst time was 50.10 seconds. This time was achieved by someone with very little or no IT skills. It is important to note that the sum of all times resulted in an incredible 34.089 seconds. This time is much faster than using the Polkadot playground user interface. This is due to the simplicity of application requirements for users. In the Polkadot, the playground user has to describe many other parameters, for example, an exact route with junctions from origin to destination. At the same time, the solution only specifies a few basic parameters such as destination or origin Parachain, destination account, amount of currency, and similar. The second test showed positive results regarding time gaps between different IT skills of users. The results can be seen in Table I. As the Table explains, the best time to send a transaction was 22.28 seconds. This time was achieved by the user with the most IT skill. This means that experienced users can adapt to new application relatively quickly. The worst time was 50.10 seconds. This time was achieved by someone with very little or no IT skills. It is important to note that the sum of all times resulted in an incredible 34.089 seconds. This time is much faster than using the Polkadot playground user interface. This is due to the simplicity of application requirements for users. In the Polkadot, the playground user has to describe many other parameters, for example, an exact route with junctions from origin to destination. At the same time, the solution only specifies a few basic parameters such as destination or origin Parachain, destination account, amount of currency, and similar.

VI. Conclusion and Future Work

Focus was to create a solution that allows users to connect substrate-compatible nodes, transfer tokens or messages between them quickly, and delete this connection when needed. The proposed solution fixes the trust problems faced by the

TABLE I
TEST 2 - EXECUTE TRANSACTION - RESULTS

Test num.	Time (s)
1.	22.28
2.	23.17
3.	25.94
4.	27.74
5.	29,02
6.	30.75
7.	39.14
8.	43.77
9.	48.98
10.	50.10
Average	**34,089**

non-native asset-sharing segment in blockchain interoperability. Application tests were designed to determine whether transactions are fast enough and if the user can make transactions easily. According to the tests, the application has positive results. The first test showed that transactions take only ten to thirty-five seconds to process on average. The second test found that users are very fast in creating transactions and their times do not differ that much even when their information system knowledge is not on the same level. This shorter task time is essential when using services.

In the future, the solution can be implemented in a real network and serve in real situations. The solution is expected to improve continuously as the code is open source. Moreover, with the future arrival of the XCMP protocol, the application can be upgraded from HRMP to XCMP to release the Relay chain from additional data that HRMP has to store.

ACKNOWLEDGMENT

This publication was created thanks to support under the Operational Program Integrated infrastructure for the projects: International center of excellence for research on intelligent and secure information and communication technologies and systems - II. stage, ITMS 313021W404, co-financed by the ERDF. This work was also supported by the Slovak Research and Development Agency under Contract no. APVV-20-0338.

REFERENCES

[1] *Anlink chain proposal paper*. URL: https://bit.ly/3Iv7S5T (visited on 03/21/2022).

[2] *Autobahn network official website*. URL: https://www.autobahn.network/ (visited on 03/21/2022).

[3] *Basilisk network configuration file*. URL: https://github.com/galacticcouncil/Basilisk-node/blob/master/rococo-local/basilisk-karura.json (visited on 05/20/2022).

[4] *Basilisk node from Galactic council*. URL: https://github.com/galacticcouncil/Basilisk-node (visited on 05/20/2022).

[5] *Blockchain bridges*. URL: https://ethereum.org/en/bridges/ (visited on 03/28/2022).

[6] Michael Borkowski et al. "Cross-blockchain technologies: Review, state of the art, and outlook". In: *White paper*. 2019.

[7] *Collator explanation by Polkadot*. URL: https://wiki.polkadot.network/docs/learn-collator (visited on 03/20/2022).

[8] *Cosmos network official website*. URL: https://cosmos.network/ (visited on 03/21/2022).

[9] *Cumulus guide on connecting Parachain to Relay chain*. URL: https://docs.substrate.io/tutorials/v3/cumulus/connect-parachain/ (visited on 03/20/2022).

[10] *Karara node from Acala network*. URL: https://github.com/AcalaNetwork/Acala (visited on 05/20/2022).

[11] *Kintsugi official webpage*. URL: https://bridge.interlay.io/ (visited on 03/13/2022).

[12] Weiwei Liu et al. "A Vickrey auction modeled decentralized cross-blockchain protocol". In: *Journal of Systems Architecture* 117 (2021), p. 102102.

[13] *Moonbeam bridge to Ethereum*. URL: https://docs.moonbeam.network/builders/integrations/bridges/eth/chainbridge/ (visited on 03/14/2022).

[14] Satoshi Nakamoto. "Bitcoin: A peer-to-peer electronic cash system". In: *Decentralized Business Review* (2008), p. 21260.

[15] *Octopus network official website*. URL: https://oct.network/ (visited on 03/14/2022).

[16] Babu Pillai et al. "The Burn-to-Claim cross-blockchain asset transfer protocol". In: *International Conference on Engineering of Complex Computer Systems (ICECCS)*. 2020, pp. 119–124. DOI: 10.1109/ICECCS51672.2020.00021.

[17] *Polkadot crosschain protocol explanation website*. URL: https://wiki.polkadot.network/docs/learn-crosschain (visited on 03/13/2022).

[18] *Polkadot explanation of Parachains*. URL: https://wiki.polkadot.network/docs/learn-parachains (visited on 03/20/2022).

[19] *Polkadot network official website*. URL: https://polkadot.network/ (visited on 03/21/2022).

[20] *Polkadot official github page*. URL: https://github.com/paritytech/polkadot (visited on 03/20/2022).

[21] Ilham A. Qasse, Manar Abu Talib, and Qassim Nasir. "Inter Blockchain Communication: A Survey". In: *ArabWIC Annual International Conference Research Track*. ArabWIC 2019. Rabat, Morocco: ACM, 2019.

[22] *RSK official website*. URL: https://www.rsk.co/ (visited on 03/21/2022).

[23] *Rust language overview*. URL: https://www.rust-lang.org/ (visited on 03/20/2022).

[24] Viktor Valaštín et al. "Blockchain Based Car-Sharing Platform". In: *2019 International Symposium ELMAR*. 2019, pp. 5–8. DOI: 10.1109/ELMAR.2019.8918650.

[25] Yan Wang et al. "Virtual chain: A storage model supporting cross-blockchain transaction". In: *Concurrency and Computation: Practice and Experience* (2020).

[26] Xingtang Xiao et al. "A Multi-blockchain Architecture Supporting Cross-Blockchain Communication". In: *Artificial Intelligence and Security*. Singapore: Springer Singapore, 2020, pp. 592–603.